

EE5176 Assignment 1: Basics of Imaging Demosaicing, White Balancing, and Tone Mapping

Amritam Halder EE23B004

September 16, 2026

1 Introduction

Raw images captured by digital cameras using a Color Filter Array (CFA) contain only one color value per pixel location. The process of reconstructing a full-color image from this undersampled data is known as demosaicing. This assignment explores various demosaicing approaches followed by color correction and enhancement techniques.

The workflow consists of: 1. Demosaicing raw Bayer data to reconstruct RGB images 2. Applying white balancing to correct color casts 3. Applying tone mapping to enhance dynamic range and contrast

Two sets of raw images were processed: `rawimage1`, `rawimage2`, and `rawimage3` for general processing, and `kodim19` for detailed analysis with ground truth comparison.

Note: The code and the output is in the jupyter notebook "PA1_EE23B004.ipynb"

2 Question 1: Demosaicing

2.1 (a) Bilinear Interpolation

Bilinear interpolation was performed on `rawimage1.npy` using the CFA mask `bayer1.npy` (pattern: `rggb`). For each color channel (R, G, B), missing pixel values were interpolated using the `scipy.interpolate.RegularGridInterpolator` with `method='linear'` on the known samples of that channel.

The implementation preserved known pixel values exactly while estimating missing values through linear interpolation of neighboring samples of the same color. The reconstructed image was displayed alongside the original raw Bayer mosaic for comparison.

```
1 rgb_bilinear = reconstruct_rgb(rawimage1, bayer1, method="linear")
```

2.2 (b) Bicubic Interpolation

Bicubic interpolation was applied using the same procedure but with `method='cubic'`. The reconstruction was compared to the bilinear result using Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR) metrics. A difference image was computed to visualize the discrepancies between the two methods.

```

1 rgb_bicubic = reconstruct_rgb(rawimage1, bayer1, method="cubic")
2 print_metrics("Bicubic versus bilinear", rgb_bicubic, rgb_bilinear)

```

Observation: Bicubic interpolation generally produces smoother gradients and better preserves fine details in smooth regions, while showing larger differences near edges and high-frequency transitions compared to bilinear interpolation.

2.3 (c) OpenCV Demosaicing

OpenCV's built-in demosaicing function `cv2.cvtColor` with the flag `COLOR_BayerRG2RGB` was used to demosaic `rawimage1`. The raw image was first converted to `uint8` format before applying the color conversion.

```

1 rawimage1_uint8 = as_uint8(rawimage1)
2 rgb_opencv = cv2.cvtColor(rawimage1_uint8, cv2.COLOR_BayerRG2RGB).
    astype(np.float64)

```

The OpenCV result was compared against both bilinear and bicubic reconstructions using the quantitative metrics.

2.4 (d) Interpolation Assumptions and Limitations

The interpolation methods rely on three key assumptions:

1. Each color channel is locally spatially smooth, allowing nearby samples to represent missing values
2. Color and intensity do not change abruptly within an interpolation neighborhood
3. The CFA pattern and its alignment are known correctly, with no severe clipping or noise domination

When these assumptions fail:

- At color or intensity edges, channels may be interpolated from different sides, causing zippering artifacts and false colors
- Linear interpolation tends to blur edges, while cubic interpolation can produce halos/ringing or overshoot
- Incorrect CFA alignment leads to systematic color swapping throughout the image
- Noise or clipping effects are propagated to missing pixel locations
- Boundary pixels are less accurate due to lack of external samples (mitigated by clamping queries at image boundaries)

2.5 (e) Kodim19 Bicubic Interpolation

The `kodim19_raw.npy` image (with CFA pattern `rggb` and mask `kodim19_cfa.npy`) was demosaiced using bicubic interpolation to reconstruct a full-color RGB image.

```

1 rgb_kodim19_bicubic = reconstruct_rgb(kodim19_raw, kodim19_cfa, method=
    "cubic")

```

2.6 (f) YCrCb Chrominance Filtering

The bicubic reconstructed image was converted to YCrCb color space. Median filtering with a 3×3 kernel was applied separately to the Cr and Cb (chrominance) channels, while the Y (luminance) channel remained unchanged. The filtered image was then converted back to RGB.

```
1 CHROMA_FILTER_SIZE = 3
2 kodim19_ycrCb = cv2.cvtColor(kodim19_rgb_uint8, cv2.COLOR_RGB2YCrCb)
3 kodim19_ycrCb_filtered = kodim19_ycrCb.copy()
4 kodim19_ycrCb_filtered[:, :, 1] = cv2.medianBlur(kodim19_ycrCb[:, :,
    1], CHROMA_FILTER_SIZE)
5 kodim19_ycrCb_filtered[:, :, 2] = cv2.medianBlur(kodim19_ycrCb[:, :,
    2], CHROMA_FILTER_SIZE)
6 rgb_kodim19_filtered = cv2.cvtColor(kodim19_ycrCb_filtered, cv2.
    COLOR_YCrCb2RGB).astype(np.float64)
```

Justification for 3×3 filter:

- It is the smallest odd-sized window capable of removing isolated speckle-like noise
- Larger windows would begin to blur genuine color transitions, causing *color-edge bleeding* (smearing of color information across sharp edges)
- The small window preserves edge sharpness in chrominance while suppressing noise, leaving luminance untouched to retain detail

2.7 (g) Ground Truth Comparison

The demosaiced results from parts (e) and (f) were compared against the provided ground truth image `kodim19_gt.png`. Quantitative metrics (MSE and PSNR) were computed, and qualitative assessment was performed through side-by-side visualization and error maps.

```
1 print_metrics("Bicubic versus ground truth", rgb_kodim19_bicubic,
    gt_rgb)
2 print_metrics("Chroma-filtered versus ground truth",
    rgb_kodim19_filtered, gt_rgb)
```

Observations:

- The bicubic reconstruction preserves fine detail but may exhibit false color artifacts near sharp edges due to independent channel interpolation
- Chrominance filtering reduces isolated color speckles and may lower error in smooth regions
- However, filtering can also soften or desaturate fine chromatic detail, making the numerical improvement image-dependent
- Error maps show that differences are concentrated around edges and textured regions

3 Question 2: White Balancing and Tone Mapping

Before applying white balancing and tone mapping, `rawimage2` and `rawimage3` were demosaiced using bilinear interpolation (reusing the demosaicing code from Question 1), as their CFA patterns are `grbg` and `rggb` respectively.

3.1 (a) Average Gray Scene Assumption

This method assumes that the average color of the entire scene is gray. For each demosaiced image:

1. Compute the average RGB values across all pixels
2. Calculate gains as the inverse of the average values (normalized to preserve mean brightness)
3. Apply the gains to each color channel

```
1 gains1 = compute_wb_gains_average(rgb1_demosaic)
2 gains2 = compute_wb_gains_average(rgb2_demosaic)
3 gains3 = compute_wb_gains_average(rgb3_demosaic)
4 rgb1_wb_avg = apply_wb_gains(rgb1_demosaic, gains1)
5 rgb2_wb_avg = apply_wb_gains(rgb2_demosaic, gains2)
6 rgb3_wb_avg = apply_wb_gains(rgb3_demosaic, gains3)
```

Results were displayed in a 2×3 grid showing original demosaiced images (top row) and white-balanced images (bottom row) for all three raw images.

3.2 (b) Specular Highlight Assumption

This method assumes that the brightest pixel in the image corresponds to a specular reflection and should therefore be white. Using the coordinates provided in the metadata:

- `rawimage1`: (813, 829)
- `rawimage2`: (279, 1164)
- `rawimage3`: (674, 174)

For each image:

1. Extract the RGB value at the specified coordinate
2. Compute gains such that this pixel becomes `[1,1,1]` after scaling (normalized to maintain average gain of 1)
3. Apply the gains to the entire image

```
1 coord1 = (813, 829) # From metadata
2 gains1_spec = compute_wb_gains_from_image(rgb1_demosaic, coord1)
3 rgb1_wb_spec = apply_wb_gains(rgb1_demosaic, gains1_spec)
```

The same procedure was applied to `rawimage2` and `rawimage3` with their respective coordinates.

3.3 (c) Neutral Region Assumption

This method assumes that a specific known neutral (gray) region in the scene should appear gray in the final image. Using the neutral coordinates from metadata:

- rawimage1: (434, 1999)
- rawimage2: (714, 444)
- rawimage3: (564, 1549)

The procedure is identical to the specular highlight method but uses the neutral region coordinates instead.

```
1 coord1_neutral = (434, 1999) # From metadata
2 gains1_neutral = compute_wb_gains_from_image(rgb1_demosaic,
3 coord1_neutral)
3 rgb1_wb_neutral = apply_wb_gains(rgb1_demosaic, gains1_neutral)
```

Results for all three methods were displayed in 2×3 grids for visual comparison.

3.4 (d) Tone Mapping

Tone mapping was applied to the white-balanced images from the average gray method (Q2a) using two techniques:

3.4.1 Histogram Equalization

Applied independently to each channel (R, G, B) using `cv2.equalizeHist` to enhance contrast by spreading out the most frequent intensity values.

```
1 def histogram_equalization(rgb_image):
2     b, g, r = cv2.split(rgb_image)
3     b_eq = cv2.equalizeHist(b)
4     g_eq = cv2.equalizeHist(g)
5     r_eq = cv2.equalizeHist(r)
6     return cv2.merge((b_eq, g_eq, r_eq))
```

3.4.2 Gamma Correction

Applied with gamma values of 0.5, 0.7, and 0.9 using the formula:

$$V_{out} = 255 \times \left(\frac{V_{in}}{255} \right)^\gamma$$

```
1 def gamma_correction(rgb_image, gamma):
2     normalized = rgb_image.astype(np.float64) / 255.0
3     corrected = np.power(normalized, gamma)
4     return np.clip(corrected * 255.0, 0, 255).astype(np.uint8)
```

Results were displayed in a 3×5 grid (three images × five processing methods: original WB, histogram equalization, gamma=0.5, gamma=0.7, gamma=0.9) for qualitative comparison.

4 References

The implementation was based on the assignment requirements and utilized only the permitted libraries:

- NumPy for numerical operations
- SciPy for interpolation
- Matplotlib for visualization
- PIL/Pillow for image loading
- OpenCV for color space conversions and specific operations
- Stack Exchange and Geeks for Geeks for different ideas on the implementation

5 Self Declaration

I, Amritam Halder, swear on my honour that I have prepared and written the answers for this assignment and associated code by myself and have not copied from the internet, any LLM's output, or other students.